



SDEV2171

Lesson 09: State and Props in React Native

A LEADING POLYTECHNIC COMMITTED TO YOUR SUCCESS

Lesson Objectives

By the end of this lesson, students should be able to:

- Explain the difference between component state and props
- Identify situations where local state is needed for dynamic UI behaviour
- Pass data into a simple reusable child component with props
- Update local state and trigger UI changes
- Debug common state, props, and rendering issues

Lesson Roadmap

1. Reconnect lesson 07 interaction to formal state language
2. Define state versus props
3. Analyze one parent-child example
4. Guided build with one extracted child component
5. Testing and troubleshooting
6. Checkpoint and lesson-10 handoff

Starting Point From Lesson 07 Or 08

- use the same Expo Router project from the previous module work
- confirm the home screen still opens the detail screen
- confirm the detail screen still accepts input and shows visible feedback

If your project is not ready

- if your project is `configured` , keep using your own project
- if your project is `partial` or `blocked` , follow your recorded remediation steps first
- if recovery will take too long, use the instructor-guided reference path and record your next step

Why Formal State Matters Now

- lesson 07 used a small memory helper so the screen could remember changing values
- today we name that helper formally: local component state
- once a screen owns changing data, it can also pass that data down to simpler child components
- this creates a clear parent-child flow instead of one oversized screen

State Versus Props

Concept

Plain-language job

state

data a component owns and can update

props

data a parent component passes into a child

Short rule

- the parent owns the changing value
- the child receives the current value
- when the parent state changes, the child gets new props on the next render

Mental Model For Today

1. the detail screen owns the changing values
2. the user types or presses a button
3. the detail screen updates its state
4. a child preview card receives the latest values as props
5. the child component output changes

Important boundary

- the child component reads props
- the child component does not rewrite the parent state directly

Minimum State Pattern

```
import { useState } from 'react';

const [draftName, setDraftName] = useState('');
const [statusText, setStatusText] = useState('No preview yet.');
```

What this means

- the screen owns these changing values
- user actions can update them
- React re-renders the screen after state changes

Minimum Props Pattern

```
function PreviewCard({ name, statusText }) {  
  return (  
    <View>  
      <Text>{name}</Text>  
      <Text>{statusText}</Text>  
    </View>  
  );  
}  
  
<PreviewCard name={profileName} statusText={statusText} />
```

What this means

- `PreviewCard` receives values from its parent
- the child renders what it receives
- if the parent sends new values, the child output updates

One-Direction Flow

- state lives in the parent screen
- props move downward into the child
- screen updates start with a state change in the parent

Debugging reminder

- if the child output is wrong, first check the parent state
- then check the prop names
- then check how the child renders those props

Guided Build Order

1. confirm the lesson-07 or lesson-08 baseline
2. open `src/app/details.js`
3. name the changing values as state
4. decide which values stay in parent state and which values the child only needs to receive as props
5. extract one child component such as `src/components/StatePreviewCard.js`
6. pass the display values into the child with props
7. test typing, pressing, and child updates
8. keep the clearest working version

Quick Sort Before You Edit

Parent state candidates

- current input text
- final preview name
- status message after the button press

Child prop candidates

- the final preview name to display
- the status message to display

Decision rule

- if the value changes because of user actions, the parent should usually own it
- if the child only needs to show the value, pass it down as a prop

Extraction Example

Parent file: `src/app/details.js`

```
import StatePreviewCard from '../components/StatePreviewCard';

<StatePreviewCard
  name={profileName}
  statusText={statusText}
/>
```



Child file: `src/components/StatePreviewCard.js`

```
import { Text, View } from 'react-native';

export default function StatePreviewCard({ name, statusText }) {
  return (
    <View>
      <Text>{name}</Text>
      <Text>{statusText}</Text>
    </View>
  );
}
```

What to notice

Visible Target For Today

- the app still opens the home and detail routes correctly
- the detail screen owns at least two simple state values
- one child component receives props from the detail screen
- the child output changes after the parent state changes
- the back path still works

Common State And Props Mistakes

- creating the child component before the baseline screen works
- forgetting to import `useState`
- passing a prop with one name and reading a different name in the child
- trying to update child output without updating parent state
- changing too many things before retesting

Recovery Path If The Build Is Blocked

- `configured` : state updates work and the child component renders the new props correctly
- `partial` : the state or prop flow exists, but one part still fails
- `blocked` : the state-and-props flow cannot be completed during class

If you are blocked

- compare your own `src/app/details.js` to the reference app
- then compare your child component file to the reference child component
- repair one part at a time in this order:
 - first: state import and state values
 - second: handler function
 - third: prop passing in the parent
 - fourth: prop names and rendering in the child

Checkpoint

Students should be ready to show

- a detail screen with named state values
- one child component receiving props
- one visible UI change after a state update

Students should also be able to answer

- Which values belong to the parent state?
- Which values does the child receive as props?
- What changed on screen after the state update?

What We Learned Today

- lesson 07 already used a first small example of local state
- state is data a component owns and updates
- props are values a parent passes into a child
- parent state changes can drive child UI updates cleanly
- this prepares us for dynamic lists in lesson 10

Next Step

- keep the lesson-09 screen in a known-good state
- make sure the child component still renders correctly after edits
- be ready to use the same state mental model with arrays and dynamic list output next lesson